

CKA - Application Lifecycle Management 섹션 핵심 정리

1. Rolling Updates and Rollbacks

Deployment가 새 버전을 배포하거나 이전 버전으로 되돌리는 방식.

배포 전략 2가지

Recreate: 기존 Pod 전부 죽이고 새 Pod 전부 띄움 → 다운타임 발생 (잘 안 씀)

RollingUpdate: Pod를 하나씩 순차 교체 → 서비스 중단 없음 (기본값)

```
spec:
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1      # 최대 몇 개 더 띄울 수 있는지 (업데이트 중 초과 허용)
      maxUnavailable: 1 # 최대 몇 개까지 동시에 내려도 되는지
```

주요 명령어

```
# 이미지 업데이트 (롤링 업데이트 트리거)
kubectl set image deployment/myapp nginx=nginx:1.18

# 배포 진행 상황 확인
kubectl rollout status deployment/myapp

# 배포 이력 확인
kubectl rollout history deployment/myapp

# 직전 버전으로 롤백
kubectl rollout undo deployment/myapp

# 특정 버전으로 롤백
kubectl rollout undo deployment/myapp --to-revision=2

# 배포 일시 중지 / 재개
kubectl rollout pause deployment/myapp
kubectl rollout resume deployment/myapp
```

배포 이력에 변경 이유 남기기

```
kubectl set image deployment/myapp nginx=nginx:1.18 --record
# --record 는 deprecated 됐으나 아직 동작함
# 대신 annotation으로 직접 남기는 방식 권장:
kubectl annotate deployment/myapp kubernetes.io/change-cause="nginx 1.18로 업그레이드"
```

2. Commands and Arguments in Docker

Docker 이미지의 기본 실행 명령어 관련 개념. 쿠버네티스 Commands/Arguments 이해의 전제.

```
FROM ubuntu
ENTRYPOINT ["sleep"] # 컨테이너 시작 시 실행할 명령어 (고정)
CMD ["5"]           # ENTRYPOINT에 전달할 기본 인자 (덮어쓰기 가능)
```

```
docker run ubuntu-sleeper          # sleep 5 실행
docker run ubuntu-sleeper 10       # sleep 10 실행 (CMD 덮어쓰기)
docker run --entrypoint sleep2.0 ubuntu-sleeper 10 # ENTRYPOINT도 덮어쓰기
```

Dockerfile	역할	덮어쓰기
ENTRYPOINT	실행할 명령어 고정	--entrypoint 옵션으로 가능
CMD	기본 인자 (ENTRYPOINT 없으면 명령어 자체)	실행 시 인자 전달로 가능

3. Commands and Arguments in Kubernetes

Docker의 ENTRYPOINT/CMD를 쿠버네티스 Pod yaml에서 재정의하는 방법.

```
spec:
  containers:
  - name: ubuntu-sleeper
    image: ubuntu-sleeper
    command: ["sleep2.0"] # Docker의 ENTRYPOINT 덮어쓰기
    args: ["10"]         # Docker의 CMD 덮어쓰기
```

매핑 관계

Docker	Kubernetes
ENTRYPOINT	command
CMD	args

헛갈리는 포인트: 쿠버네티스의 `command` 가 Docker의 `CMD` 가 아니라 `ENTRYPOINT` 에 해당함. 이름이 달라서 자주 혼동함.

4. Configure Environment Variables in Applications

Pod 컨테이너에 환경변수 주입하는 방법.

직접 지정 (plain key-value)

```
spec:
  containers:
  - name: myapp
    image: myapp
    env:
    - name: APP_COLOR
      value: "blue"
    - name: APP_MODE
      value: "prod"
```

ConfigMap에서 가져오기

```
env:
- name: APP_COLOR
  valueFrom:
    configMapKeyRef:
      name: my-config
      key: APP_COLOR
```

Secret에서 가져오기

```
env:
- name: DB_PASSWORD
  valueFrom:
    secretKeyRef:
```

```
name: my-secret
key: DB_PASSWORD
```

5. Configuring ConfigMaps in Applications

민감하지 않은 설정값을 Pod와 분리해서 관리하는 오브젝트.

생성 방법

```
# Imperative (명령어로 직접)
kubectl create configmap my-config \
  --from-literal=APP_COLOR=blue \
  --from-literal=APP_MODE=prod

# 파일에서
kubectl create configmap my-config --from-file=app.properties
```

```
# Declarative (yaml)
apiVersion: v1
kind: ConfigMap
metadata:
  name: my-config
data:
  APP_COLOR: blue
  APP_MODE: prod
```

Pod에서 사용하는 3가지 방법

① 특정 키만 환경변수로

```
env:
- name: APP_COLOR
  valueFrom:
    configMapKeyRef:
      name: my-config
      key: APP_COLOR
```

② ConfigMap 전체를 환경변수로

```
envFrom:
- configMapRef:
  name: my-config
```

③ 볼륨으로 마운트 (파일로 읽기)

```
volumes:
- name: config-volume
  configMap:
    name: my-config
containers:
- volumeMounts:
  - mountPath: /etc/config
    name: config-volume
```

```
kubectl get configmap
kubectl describe configmap my-config
```

6. Secrets

민감한 정보(비밀번호, API키, 인증서 등)를 저장하는 오브젝트. ConfigMap과 구조는 같지만 **base64 인코딩**으로 저장됨.

주의: base64는 암호화가 아님. 누구나 디코딩 가능. 진짜 보안은 etcd 암호화(7번) + RBAC으로 접근 제어해야 함.

생성 방법

```
# Imperative
kubectl create secret generic my-secret \
  --from-literal=DB_PASSWORD=mysecretpassword \
  --from-literal=DB_USER=admin

# 파일에서
kubectl create secret generic my-secret --from-file=ssh-privatekey
```

```
# Declarative (값을 직접 base64 인코딩해서 넣어야 함)
apiVersion: v1
```

```
kind: Secret
metadata:
  name: my-secret
type: Generic
data:
  DB_PASSWORD: bXlZZWNyZXRwYXNzd29yZA== # base64 인코딩된 값
  DB_USER: YWRtaW4=
```

```
# base64 인코딩/디코딩
echo -n "mysecretpassword" | base64
echo -n "bXlZZWNyZXRwYXNzd29yZA==" | base64 --decode
```

Pod에서 사용하는 3가지 방법

① 특정 키만 환경변수로

```
env:
- name: DB_PASSWORD
  valueFrom:
    secretKeyRef:
      name: my-secret
      key: DB_PASSWORD
```

② Secret 전체를 환경변수로

```
envFrom:
- secretRef:
    name: my-secret
```

③ 볼륨으로 마운트

```
volumes:
- name: secret-volume
  secret:
    secretName: my-secret
containers:
- volumeMounts:
  - mountPath: /etc/secrets
    name: secret-volume
    readOnly: true
```

Secret type 종류

type	용도
generic	일반 key-value
docker-registry	프라이빗 레지스트리 인증
tls	TLS 인증서/키

```
kubectl get secret
kubectl describe secret my-secret # 값은 안 보이고 크기만 표시됨
kubectl get secret my-secret -o yaml # base64 인코딩된 값 확인 가능
```

7. Demo: Encrypting Secret Data at Rest

Secret은 기본적으로 etcd에 **평문(base64만 된 상태)**으로 저장됨 → etcd에 직접 접근하면 값을 볼 수 있음 → **EncryptionConfiguration**으로 etcd에 저장 시 실제 암호화 적용 가능.

암호화 설정 흐름

① EncryptionConfiguration 파일 작성

```
apiVersion: apiserver.config.k8s.io/v1
kind: EncryptionConfiguration
resources:
- resources:
  - secrets
providers:
- aescbc:
  keys:
  - name: key1
    secret: <base64로 인코딩된 32바이트 키>
- identity: {} # 암호화 안 된 기존 값도 읽을 수 있게 fallback
```

② kube-apiserver에 옵션 추가

```
# /etc/kubernetes/manifests/kube-apiserver.yaml
spec:
  containers:
  - command:
    - kube-apiserver
    - --encryption-provider-config=/etc/kubernetes/enc/encryption-config.yaml
```

③ 확인

```
# etcd에서 직접 조회해서 암호화 됐는지 확인
ETCDCTL_API=3 etcdctl get /registry/secrets/default/my-secret \
  --cacert=... --cert=... --key=... | hexdump -C
# 암호화 됐으면 k8s:enc:aescbc:... 로 시작하는 바이트 보임
```

기존에 저장된 Secret은 자동으로 암호화되지 않음. 재저장 필요:

```
kubectl get secrets -A -o json | kubectl replace -f -
```

8. Multi Container Pods

하나의 Pod 안에 **여러 컨테이너**를 함께 실행하는 것.

- 같은 Pod 안 컨테이너들은 네트워크(localhost), 스토리지(volume) 공유
- 생명주기 공유: Pod가 뜨면 같이 뜨고, 죽으면 같이 죽음
- 가장 흔한 패턴: 메인 앱 컨테이너 + 보조 역할 컨테이너(사이드카)

```
spec:
  containers:
  - name: app
    image: myapp
    ports:
    - containerPort: 8080
  - name: log-agent          # 사이드카 컨테이너
    image: log-agent
    volumeMounts:
    - mountPath: /var/log
      name: shared-logs
  volumes:
  - name: shared-logs
    emptyDir: {}
```

멀티 컨테이너 Pod에서 특정 컨테이너 로그 보기

```
kubectl logs 파드이름 -c 컨테이너이름
```

특정 컨테이너에 exec 접속

```
kubectl exec -it 파드이름 -c 컨테이너이름 -- /bin/sh
```

9. Multi Container Pods Design Pattern

멀티 컨테이너 Pod의 대표적인 3가지 패턴.

Sidecar (사이드카)

메인 앱을 보조하는 컨테이너.

앱 컨테이너 → 로그 파일에 기록
사이드카 → 그 로그 파일을 읽어서 중앙 로그 서버로 전송

가장 흔한 패턴. 앱 코드 수정 없이 로깅/모니터링/보안 기능을 붙일 수 있음.

Adapter (어댑터)

메인 앱의 출력을 **표준 포맷으로 변환**하는 컨테이너.

앱 컨테이너 → 고유한 포맷으로 로그 출력
어댑터 → 그걸 Prometheus 같은 모니터링 시스템이 읽을 수 있는 포맷으로 변환

Ambassador (앰배서더)

메인 앱 대신 **외부 연결을 대리**하는 컨테이너.

앱 컨테이너 → localhost:3306 으로 DB 접속 요청
앰배서더 컨테이너 → 환경(dev/staging/prod)에 맞는 실제 DB로 연결 대리

앱 코드에서 환경별 DB 주소를 신경 안 써도 됨.

10. InitContainers

메인 컨테이너가 시작하기 전에 먼저 실행되고 완료되어야 하는 초기화 전용 컨테이너.

- 순서대로 하나씩 실행 (병렬 아님)
- 하나라도 실패하면 Pod 전체가 재시작됨
- 완료되면 종료 (계속 실행되지 않음) → 사이드카와의 차이점

```
spec:
  initContainers:
  - name: init-db
    image: busybox
```

```
command: ['sh', '-c', 'until nslookup mydb; do echo waiting for db; sleep 2; done']
- name: init-config
  image: busybox
  command: ['sh', '-c', 'cp /tmp/config /etc/app/config']
containers:
- name: myapp
  image: myapp
```

언제 씬?

DB가 완전히 뜰 때까지 앱 시작 대기
설정 파일 다운로드 후 앱 시작
권한/디렉터리 초기화 후 앱 시작

```
kubectl describe pod 파드이름
# Init Containers 섹션에서 상태 확인 가능
# State: Terminated (Completed) 이면 정상 완료
```

11. Self Healing Applications

쿠버네티스가 Pod/컨테이너 장애를 **자동으로 복구**하는 메커니즘.

ReplicaSet / Deployment 수준

- Pod가 죽으면 → 자동으로 새 Pod 생성해서 replicas 수 유지
- 노드가 죽으면 → NoExecute taint 자동 적용 → Pod 다른 노드로 이동

컨테이너 수준 (Probe)

kubelet이 컨테이너 상태를 주기적으로 체크해서 문제 있으면 재시작.

Liveness Probe: 컨테이너가 살아있는지 체크. 실패하면 컨테이너 재시작.

```
LivenessProbe:
  httpGet:
    path: /healthz
    port: 8080
  initialDelaySeconds: 10 # 처음 체크까지 대기 시간
  periodSeconds: 5       # 체크 주기
  failureThreshold: 3     # 몇 번 실패하면 재시작할지
```

Readiness Probe: 컨테이너가 트래픽 받을 준비됐는지 체크. 실패하면 Service에서 해당 Pod 제외 (재시작은 안 함).

```
readinessProbe:
  httpGet:
    path: /ready
    port: 8080
  initialDelaySeconds: 5
  periodSeconds: 3
```

Startup Probe: 앱 시작 자체가 오래 걸릴 때 사용. Startup Probe가 성공하기 전까지 Liveness/Readiness Probe 실행 안 함.

```
startupProbe:
  httpGet:
    path: /healthz
    port: 8080
  failureThreshold: 30
  periodSeconds: 10
```

Probe 방식 3종류

방식	설명
httpGet	HTTP 요청 → 200-399 응답이면 성공
tcpSocket	TCP 포트 열려있으면 성공
exec	명령어 실행 → 종료코드 0이면 성공

12. Introduction to Autoscaling

쿠버네티스에서 부하에 따라 **자동으로 규모를 조절**하는 기능.

두 가지 차원

수평 확장 (HPA) → Pod 개수를 늘리거나 줄임
수직 확장 (VPA) → Pod의 CPU/메모리 requests/limits 값을 늘리거나 줄임

언제 필요?

트래픽이 갑자기 몰릴 때 → 사람이 수동으로 replicas 조정하기 힘들
밤에는 트래픽 적는데 낮에는 많을 때 → 자동으로 줄였다 늘렸다

13. Horizontal Pod Autoscaler (HPA)

CPU/메모리 사용률 등 지표를 보고 **Pod 개수를 자동 조절**하는 오브젝트.

- Metrics Server가 설치돼 있어야 작동함

생성 방법

Imperative

```
kubectl autoscale deployment myapp --cpu-percent=50 --min=1 --max=10
```

Declarative

```
apiVersion: autoscaling/v2
```

```
kind: HorizontalPodAutoscaler
```

```
metadata:
```

```
  name: myapp-hpa
```

```
spec:
```

```
  scaleTargetRef:
```

```
    apiVersion: apps/v1
```

```
    kind: Deployment
```

```
    name: myapp
```

```
  minReplicas: 1
```

```
  maxReplicas: 10
```

```
  metrics:
```

```
  - type: Resource
```

```
    resource:
```

```
      name: cpu
```

```
      target:
```

```
        type: Utilization
```

```
        averageUtilization: 50    # CPU 사용률 50% 초과 시 스케일 아웃
```

동작 방식

CPU 사용률 > 50% → Pod 개수 늘림 (Scale Out)

CPU 사용률 < 50% → Pod 개수 줄임 (Scale In)

최소 minReplicas, 최대 maxReplicas 범위 안에서 조절

```
kubectl get hpa
kubectl describe hpa myapp-hpa
```

HPA가 작동하려면 Pod에 반드시 **requests.cpu** 가 설정돼 있어야 함. 없으면 사용률 계산 불가.

14. In-Place Resize of Pods

기존에는 Pod의 CPU/메모리 requests/limits를 바꾸려면 **Pod를 재시작해야** 했음.

쿠버네티스 1.27+부터 **In-Place Pod Resize** 기능으로 재시작 없이 자원 조정 가능 (현재 alpha/beta 단계).

```
spec:
  containers:
  - name: myapp
    image: myapp
    resources:
      requests:
        cpu: "500m"
        memory: "256Mi"
      limits:
        cpu: "1"
        memory: "512Mi"
    resizePolicy:
      - resourceName: cpu
        restartPolicy: NotRequired # CPU는 재시작 없이 변경 가능
      - resourceName: memory
        restartPolicy: RestartContainer # 메모리는 변경 시 재시작 필요
```

CKA 시험에서는 개념 수준으로만 알고 있으면 됨.

15. Vertical Pod Autoscaling (VPA)

Pod의 CPU/메모리 requests/limits 값 자체를 자동으로 조절하는 오브젝트.

- 기본 쿠버네티스에 내장 안 됨 → 별도 설치 필요 (`kubernetes/autoscaler` 레포)
- 과거 사용량 데이터를 분석해서 적정 requests/limits 값을 추천하거나 자동 적용

```

apiVersion: autoscaling.k8s.io/v1
kind: VerticalPodAutoscaler
metadata:
  name: myapp-vpa
spec:
  targetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: myapp
  updatePolicy:
    updateMode: "Auto"    # Auto, Off, Initial 중 선택

```

updateMode 옵션

옵션	동작
Off	추천값만 계산, 실제 적용 안 함 (분석용)
Initial	Pod 생성 시에만 적용, 이미 뜬 Pod는 건드리지 않음
Auto	자동 적용 (현재는 Pod 재시작을 통해 적용됨)

HPA vs VPA 비교

	HPA	VPA
조절 대상	Pod 개수	Pod의 CPU/메모리 크기
주로 쓰는 경우	트래픽 변동이 큰 웹 서비스	트래픽 예측이 어렵거나 단일 Pod인 경우
같이 쓰면	CPU 기준으로 함께 쓰면 충돌 가능	CPU 제외한 메모리만 VPA로 관리하는 식으로 병용 가능

CKA 시험에서는 개념 수준으로만 알고 있으면 됨.

CKA 시험 팁 요약

- **Rolling Update/Rollback:** `kubectl rollout` 명령어 흐름 꼭 익힐 것. `rollout undo`, `rollout history` 자주 나옴.
- **command vs args:** Docker의 ENTRYPOINT/CMD와 매핑 관계 헷갈리지 않도록 주의.
- **ConfigMap vs Secret:** 구조는 동일, Secret은 base64 인코딩. `envFrom`, `env.valueFrom`, `volume` 세 가지 사용 방법 모두 숙지.
- **Secret 암호화:** etcd 저장 시 암호화 설정 흐름 (EncryptionConfiguration → kube-apiserver 옵션 추가) 알아둘 것.

- **InitContainers:** 메인 컨테이너 시작 전 완료되어야 하는 것. 실패 시 Pod 재시작.
- **Liveness vs Readiness Probe:** Liveness 실패 → 재시작, Readiness 실패 → Service에서 제외. 차이 구분 필수.
- **HPA:** `kubectl autoscale` 명령어와 yaml 구조, requests.cpu 설정 필수 조건 알아둘 것.
- **VPA, In-Place Resize:** 개념만 알면 충분.